Student Performance Analytics System

Project Documentation

1. Project Title

Student Performance Analytics System

Python tool that grades 15 students and reports class-level analytics.

2. Overview

- Command-line Python project built with **pandas** and **NumPy**
- Dataset: 15 students (IDs 501–515), 5 subjects marked out of 100, plus attendance %
- Flow: read data → total and average → grade → pass/fail → class analysis
- Output: console report and two CSV files

Technology stack

Component	Purpose
Python 3.8+	Core programming language
pandas	Data creation, CSV reading/writing, calculations
NumPy	Rounding of average values

Project structure

File	Description
`main.py`	Main script containing all logic
`student_data.csv`	Generated raw marks and attendance
`student_performance.csv`	Raw data plus Total, Average, Grade and Result
`README.md`	Usage and project notes

3. Objectives

1. Create a structured student dataset with subject marks and attendance.
2. Store the raw data in a CSV file and read it back for processing.
3. Calculate the **Total** (out of 500) and **Average** for every student.
4. Assign a **letter grade** (A+ to F) based on the average.
5. Determine a **Pass/Fail** result using a minimum-mark rule per subject.
6. Produce **class-level statistics**: class average, highest and lowest averages, and pass/fail counts.
7. Save the complete report to a CSV file for later use.

4. Implementation

4.1 Data Creation and Storage

- Marks defined in a Python dictionary and turned into a DataFrame
- Saved as `student_data.csv`
- Read back with `pd.read_csv()` and printed

4.2 Total and Average Calculation

Using the list of subjects, row-wise calculations are applied across the five subject columns:

- **Total** = sum of the five subject marks (`axis=1`)
- **Average** = mean of the five marks, rounded to 2 decimal places with `np.round()`

4.3 Grade Assignment

A function `calculate_grade(avg)` maps the average to a grade and is applied to the Average column:

Average	Grade
90 and above	A+
80 – 89	A
70 – 79	B
60 – 69	C
50 – 59	D
40 – 49	E
Below 40	F

4.4 Pass/Fail Logic

- Rule: pass only if **every** subject mark is 35 or more
- One mark below 35 means **Fail**, whatever the average
- Code: `df[subjects].min(axis=1)` with a lambda check

4.5 Class Analysis

The script prints the following summary using pandas aggregation functions:

- Class average (mean of all student averages)
- Highest and lowest average
- Number of students who passed and failed

4.6 Report Export

The final DataFrame, containing the original columns plus Total, Average, Grade and Result, is saved as `student_performance.csv`.

4.7 How to Run

```
pip install pandas numpy
python main.py
```

Running the script recreates both CSV files in the current directory and prints the full report.

5. Final Outcome

5.1 Class Analysis Results

The results below were verified against the dataset:

Metric	Value
Class Average	78.65
Highest Average	93.0 (Jyothika, ID 503)
Lowest Average	45.6 (Janu, ID 515)
Students Passed	13
Students Failed	2

5.2 Grade Distribution

Grade	Students	Count
A+	Jyothika, Prathibha, Monisha	3
A	Priya, Jaya Sree, Thejaswini, Rajeswari, Monika	5
B	Priyanka, Arifa, Zaara, Nandhini	4
C	Vaishnavi	1
D	Shalini	1
E	Janu	1
F	None	0

5.3 Key Observations

- **Top performer:** Jyothika (ID 503) with a total of 465 and an average of 93.0 (Grade A+).
- **Failed students:** Shalini (ID 513) scored 34 in JavaScript and Janu (ID 515) scored 33 in Python, so both fail even though Shalini's average (57.2) would otherwise earn a Grade D.
- **Most students perform well:** 8 of 15 students achieved an A or A+ grade.
- **Attendance** is recorded but is not used in grading. Several students have low attendance (Arifa 56%, Janu 59%, Nandhini 71%, Monika 74%).

5.4 Deliverables

- A working Python script (`main.py`)
- `student_data.csv` – raw dataset
- `student_performance.csv` – full report with Total, Average, Grade and Result
- A console report with the student table, performance report and class analysis

5.5 Limitations and Future Enhancements

- Load student data from an external file instead of hard-coding it.
- Include attendance in the result (for example, a minimum of 75% required).
- Add subject-wise statistics, topper lists and rank calculation.
- Add visualizations with matplotlib or seaborn.

5.6 Conclusion

- Automates marks processing, grading and results for a class
- Gives a clear summary of overall performance
- Shows practical pandas use: row-wise calculations, custom functions, CSV handling